



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Cultura i Esport



IES JUAN DE GARAY

C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN
Y FORMACIÓN PROFESIONAL



Unión Europea

Fondo Social Europeo
El FSE invierte en tu futuro

UD 06.3. JAVASCRIPT MEDIO CLASE

LM

Ciclos Formativos Grado Superior

Pascual Ligerio
pa.ligero2@edu.gva.es

2025/2026

Versión:260321.1833

Licencia



Reconocimiento - NoComercial - Compartirlgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚗ Interesante

ÍNDICE DE CONTENIDO

1. Funciones anónimas y Funciones flecha.....	5
1.1 Introducción.....	5
1.2 forma clásica.....	5
1.3 funciones anónimas.....	5
1.4 función flecha.....	5
1.5 Ejercicio.....	6
2. Funciones promesa.....	6
2.1 Cómo funciona una promesa.....	6
2.2 Reloj cuenta atrás.....	7
3. Fetch.....	9
3.1 Introducción.....	9
3.2 Cómo funciona fetch.....	10
3.3 Ejemplo básico: Obtener datos (GET).....	10
4. Almacenar información en el navegador.....	14
4.1 Localstorage.....	14
4.2 Ejemplo sencillo con localStorage.....	14

UD06.3. JAVASCRIPT MEDIO CLASE

1. FUNCIONES ANÓNIMAS Y FUNCIONES FLECHA

1.1 Introducción

Sobre un formulario con dos input numéricos podemos ver los 3 tipos.

Código en 01Anonimas.html

1.2 forma clásica

```
function nombre(atributos) {  
    .....  
};  
  
function sumar(a, b) {  
    return a + b;  
}
```

1.3 funciones anónimas

```
//anónima  
const sumaAnonima = function (a,b) {  
    return parseInt(a)+parseInt(b)  
}  
  
boton.addEventListener("click", function() {  
    resultado.innerHTML=`con anonima: ${sumaAnonima(num1.value, num2.value)}`  
});
```

1.4 función flecha

```
//flecha  
const sumaFlecha = (a,b) => parseInt(a)+parseInt(b);  
  
boton.addEventListener("click", function() {  
    resultado.innerHTML=`con flecha: ${sumaFlecha(num1.value, num2.value)}`  
})
```

1.5 Ejercicio

Escribe un programa, con funciones flecha, que devuelva la suma de todos los números inferiores iguales a el.

Ej:

numeroTriangular(3)= 3+2+1

2. FUNCIONES PROMESA

2.1 Cómo funciona una promesa

Una promesa se crea usando el constructor Promise, al que se le pasa una función con dos parámetros: resolve y reject.

- **resolve:** Se llama cuando la operación es exitosa. (→ .then ...)
- **reject:** Se llama cuando ocurre un error. (→ .catch ...)

```
function promesa(){
  return new Promise((resolve,reject) => {
    const exito = true; // Simula éxito o fracaso
    if (exito) {
      resolve("¡Operación exitosa!");
    } else {
      reject("Hubo un error.");
    }
  });
};

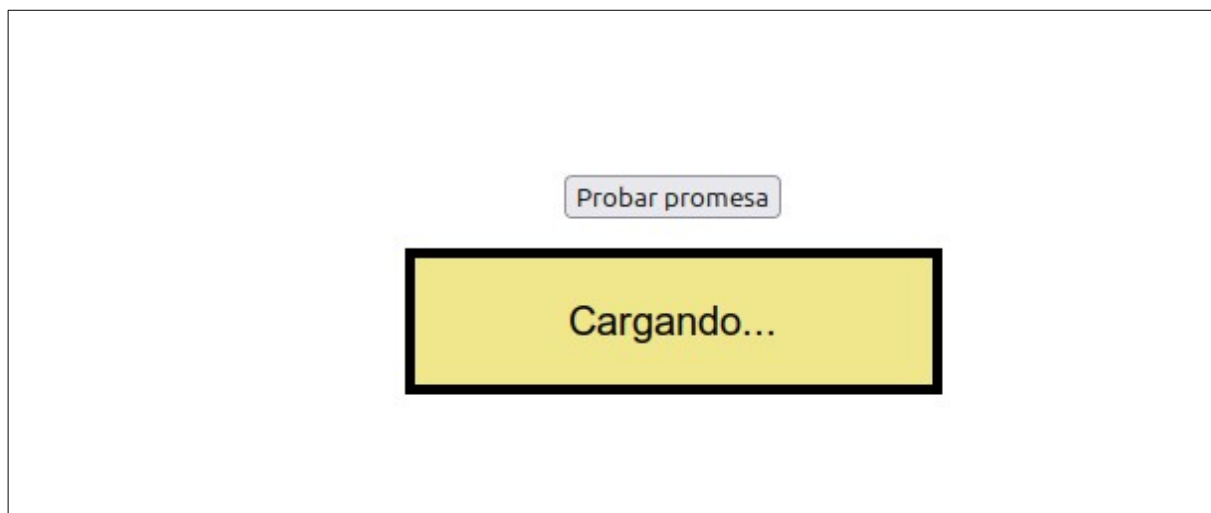
promesa()
  .then(resultado => console.log("Clásica: ",resultado)) // Maneja el éxito
  .catch(error => console.error("Clásica: ",error)); // Maneja el error
```

// Espera 2 segundos y contesta en función de la variable éxito.

```
function miPromesa() {
  return new Promise((resolve, reject) => {
    const exito = true; // Simulamos si la operación fue exitosa
    setTimeout(() => {
      if (exito) {
        resolve("Operación exitosa.");
      } else {
        reject("Ocurrió un error.");
      }
    }, 2000); // Simula una espera de 2 segundos antes de comprobar el valor de la
```

```
        variable éxito  
    });  
}
```

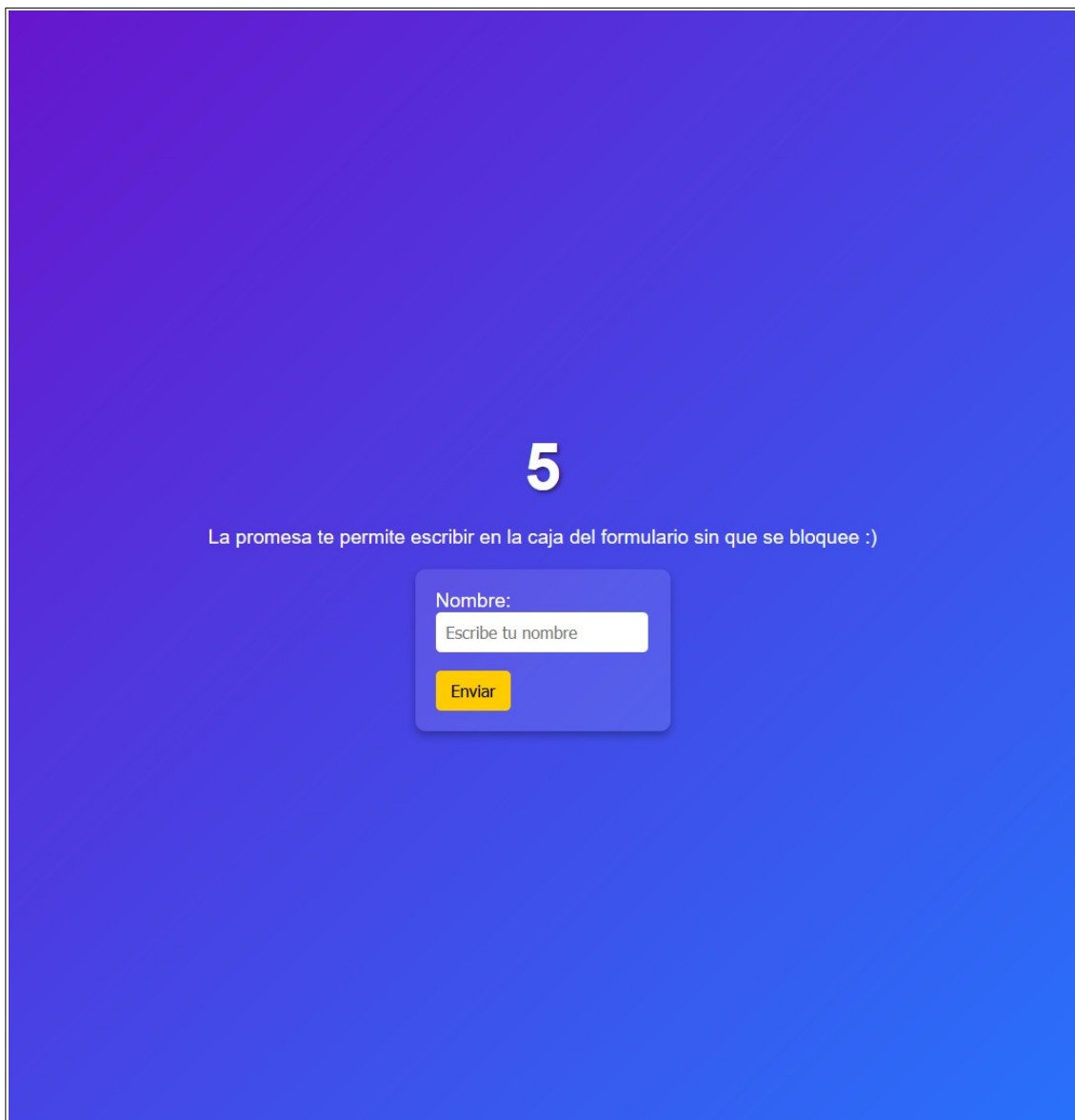
en **02Promesas.html** puedes probar un código de ejemplo :



2.2 Reloj cuenta atrás

Vamos a implementar una simple cuenta atrás, la función promesa se llama así misma cada segundo cambiando el valor de la cuenta en pantalla:

En **03Promesas.html** puedes probar un código de ejemplo :



```
<body>
  <h1 id="resultado">5</h1>
  <div>
    La promesa te permite escribir en la caja del formulario sin que se bloquee :)
  </div>
  <form id="form" action="#" method="post">
    <label for="name">Nombre:</label>
    <br>
    <input type="text" id="name" name="name" placeholder="Escribe tu nombre"
required>
    <br><br>
```

```
<button type="submit" id="enviar">Enviar</button>
</form>
```

```
<script>
  //variables
  let name=document.getElementById("name")
  // Función que maneja la cuenta regresiva usando una promesa
  function actualizarReloj() {
    return new Promise((resolve) => {
      setTimeout(() => {
        segundos--;
        document.getElementById("resultado").innerHTML = segundos;

        if (segundos > 0) {
          actualizarReloj(); // Llamada recursiva hasta llegar a cero
        } else {
          resolve("¡He terminado!");
          document.getElementById("resultado").innerHTML = "¡ACABÉ ${name.value}!";
        }
      }, 1000); // Espera de un segundo
    });
  }

  // Inicialización de la cuenta regresiva
  let segundos = 5; // Tiempo inicial en segundos
  document.getElementById("resultado").innerHTML = segundos; // Mostrar el tiempo
  inicial
  actualizarReloj(); // Iniciar la cuenta regresiva
</script>
</body>
</html>
```

3. FETCH

3.1 Introducción

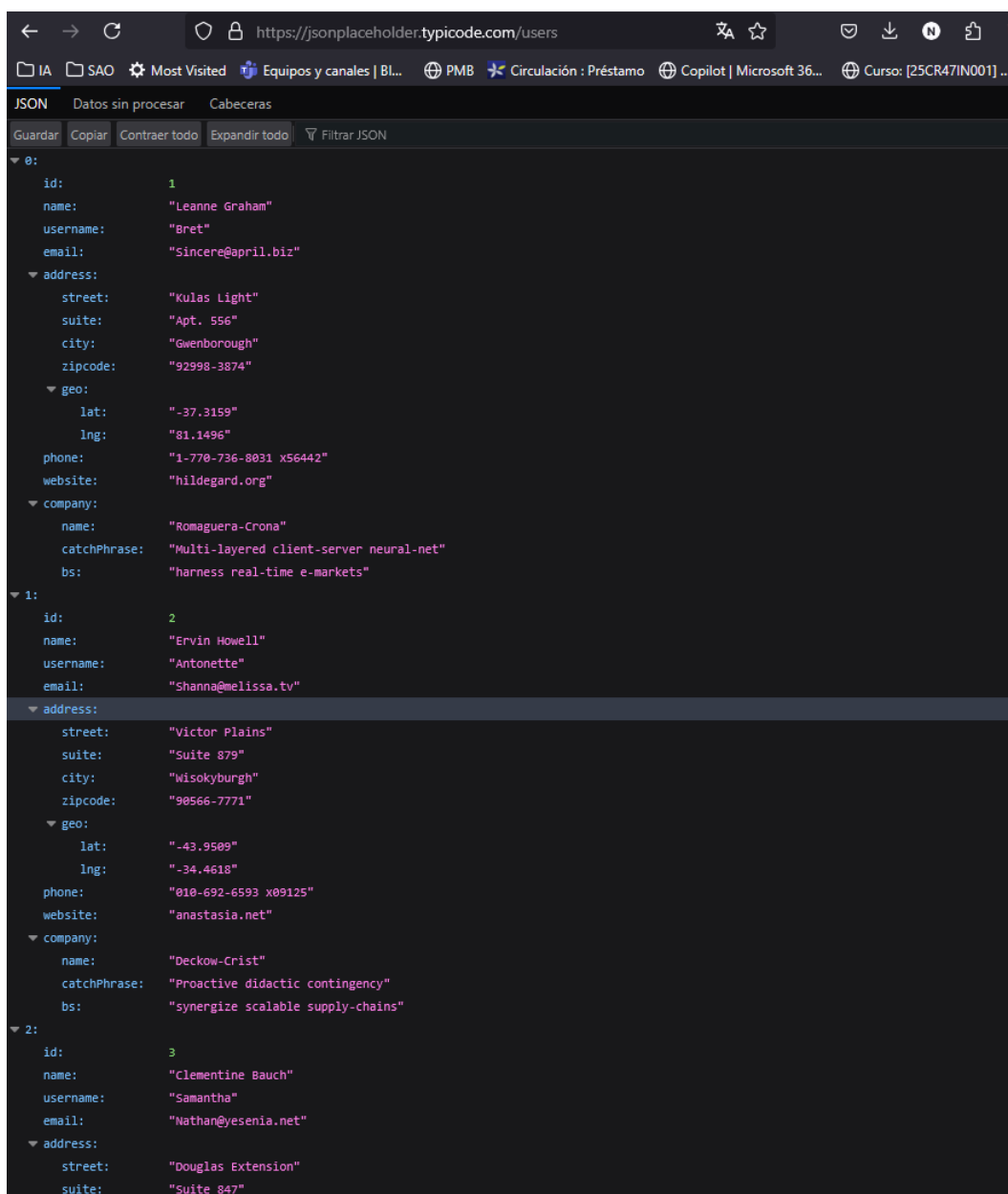
La API `fetch` es una herramienta moderna en JavaScript que te permite realizar solicitudes HTTP (como GET, POST, etc.) para obtener o enviar datos a un servidor. Es muy útil para trabajar con APIs y manejar datos en aplicaciones web.

3.2 Cómo funciona fetch

1. fetch devuelve una **promesa**. Esto significa que trabajará de manera asíncrona, esperando una respuesta del servidor.
2. Puedes usar `.then()` para manejar la respuesta cuando llegue.
3. Si hay un error (por ejemplo, el servidor no responde), puedes manejarlo con `.catch()`.

3.3 Ejemplo básico: Obtener datos (GET)

Existe un servidor de pruebas que nos permite “jugar con fetch, pon en un navegador la url <https://jsonplaceholder.typicode.com/users> y verás los datos que te devuelve la petición.



Hacemos una petición a una página y mostramos en pantalla el username y el email de cada uno.

Con fetch (1) le pedimos los datos, la url nos devuelve una cadena de texto y la convertimos en formato JSON (2), Recorremos como si fuera un array (3) y mostramos en pantalla.

Let url= <https://jsonplaceholder.typicode.com/users>

```
<body>
  <h1>Lista de Usuarios</h1>
  <p id="resultado">Cargando...</p>
  <script>
    // URL de una API pública
    const url = 'https://jsonplaceholder.typicode.com/users';
    let resultado = document.getElementById("resultado");

    function pedirDatos() {
      // Usamos fetch para hacer una solicitud GET
      fetch(url) //1
        .then(response => {
          // Verificamos si la respuesta fue exitosa
          if (!response.ok) {
            throw new Error(`Error en la solicitud: ${response.status} ${response.statusText}`);
          }
          // Convertimos la respuesta a JSON
          return response.json(); //2
        })
        .then(data => {
          // Aquí trabajamos con los datos obtenidos
          console.log(data);

          // Creamos una lista para mostrar los datos
          const lista = document.createElement('ul');
          data.forEach(usuario => { //3
            const item = document.createElement('li');
            item.className = "element";
            item.innerHTML = `<strong>Nombre:</strong> ${usuario.name} <br>
            <strong>Correo:</strong> ${usuario.email}`;
            lista.appendChild(item);
          });
          resultado.innerHTML = ""; // Limpiamos el mensaje de "Cargando..."
```

```
        resultado.appendChild(lista); // Añadimos la lista al DOM
    })
    .catch(error => {
        // Manejo de errores
        console.log('Hubo un problema con la solicitud:', error);
        resultado.innerHTML = `<span style="color:red;">${error.message}</span>`;
    });
}

// Inicio
resultado.innerHTML = "Cargando...";

// Pausa dramática antes de cargar los datos
setTimeout(() => {
    resultado.innerHTML = "Datos:";
    resultado.innerHTML += "<hr>";
    pedirDatos();
}, 2000); // Espera de 2 segundos
</script>
</body>
```

Lista de Usuarios

Nombre: Leanne Graham
Correo: Sincere@april.biz

Nombre: Ervin Howell
Correo: Shanna@melissa.tv

Nombre: Clementine Bauch
Correo: Nathan@yesenia.net

Nombre: Patricia Lebsack
Correo: Julianne.OConner@kory.org

Nombre: Chelsey Dietrich
Correo: Lucio_Hettinger@annie.ca

Nombre: Mrs. Dennis Schulist
Correo: Karley_Dach@jasper.info

Nombre: Kurtis Weissnat
Correo: Telly.Hoeger@billy.biz

Nombre: Nicholas Runolfsson
Correo: Sherwood@rosamond.me

Nombre: Glenna Reichert
Correo: Chaim_McDermott@dana.io

Nombre: Clementina DuBuque
Correo: Rey.Padberg@karina.biz

4. ALMACENAR INFORMACIÓN EN EL NAVEGADOR

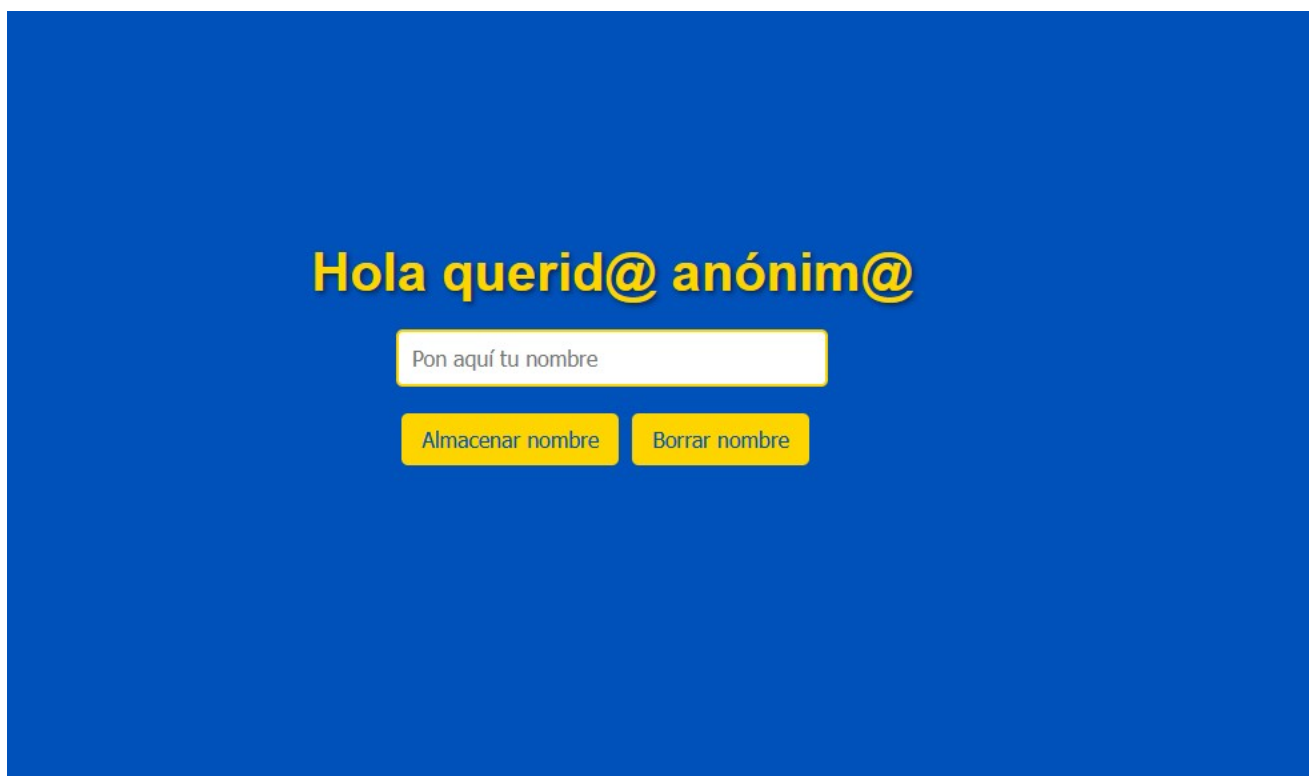
4.1 Localstorage

En el navegador, puedes almacenar información utilizando **Web Storage**, que incluye dos opciones principales: **localStorage** y **sessionStorage**. Ambas te permiten guardar datos en forma de clave-valor directamente en el navegador del usuario, pero tienen diferencias:

- **localStorage**: Almacena datos que permanecen incluso si cierras el navegador y lo vuelves a abrir.
- **sessionStorage**: Almacena datos solo mientras la pestaña del navegador está abierta. Cuando cierras la pestaña, los datos se eliminan.

4.2 Ejemplo sencillo con localStorage

Guardar el nombre de un usuario en el navegador. Que cada vez que cargues la página te salude o te diga que lo guardes.



- Guardar:

```
localStorage.setItem('nombreUsuario', nombre); // Guarda el valor con la clave nombreUsuario
```

- Recuperar:

```
const nombreGuardado = localStorage.getItem('nombreUsuario'); // Recupera el valor
```

- Eliminar:

```
localStorage.removeItem('nombreUsuario');

<body>
  <h1 id="saludo">Hola querid@ anónim@</h1>
  <input type="text" id="nombre" placeholder="Pon aquí tu nombre">
  <div class="button-container">
    <button id="boton">Almacenar nombre</button>
    <button id="botonBorrar">Borrar nombre</button>
  </div>

  <script>
    // Referencias a elementos del DOM
    let saludo = document.getElementById("saludo");
    let nombre = document.getElementById("nombre");
    let boton = document.getElementById("boton");
    let botonBorrar = document.getElementById("botonBorrar");

    // Evento para almacenar el nombre en localStorage
    boton.addEventListener("click", function() {
      localStorage.setItem("nombre", nombre.value); // Guardar el nombre
      saludo.textContent = "Hola " + nombre.value; // Actualizar el saludo
    });

    // Evento para borrar el nombre de localStorage
    botonBorrar.addEventListener("click", function() {
      localStorage.removeItem("nombre"); // Borrar el nombre
      saludo.textContent = "Hola querid@ anónim@"; // Actualizar el saludo
    });

    // Recuperar el nombre almacenado al cargar la página
    let nombreRecuperado = localStorage.getItem("nombre");
    if (nombreRecuperado) {
      saludo.textContent = "Hola " + nombreRecuperado; // Mostrar el saludo personalizado
    } else {
      saludo.textContent = "Hola querid@ anónim@"; // Mostrar el saludo por defecto
    }
  </script>

</body>
```